Java Interview Notes

Method Overriding — Complete Guide

Topics: Overriding | Why Overriding | Dynamic Dispatch | Rules | Overloading vs Overriding

1. What is Method Overriding?

Method Overriding means: jab child class apne parent class ke method ko apne hisaab se redefine karti hai — same name, same parameters, same return type.

> **Simple Definition:** Parent class ka method child class mein dobara likho — apna kaam karwane ke liye.

Real Life Example

Socho ek Animal class hai jisme sound() method hai. Ab Dog class usse inherit karti hai, lekin Dog apni awaaz bolta hai — bark. Toh Dog apne andar sound() ko override karti hai.

```
// Parent class
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class — method override kar raha hai
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks: Woof!");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows: Meow!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();   // IS-A relationship
        a.sound();             // Output: "Dog barks: Woof!"

        Animal b = new Cat();
        b.sound();             // Output: "Cat meows: Meow!"
    }
```

```
}
```

2. Why Method Overriding?

Interview mein ye question zaroor aata hai — '3 reasons yaad rakho'

Reason 1 — Runtime Polymorphism

Ek hi method call alag-alag objects pe alag-alag kaam karta hai. Yahi polymorphism hai.

Reason 2 — Code Reusability

Parent class ka sara code milta hai. Sirf jo alag hai woh method override karo. Poora naya class banana nahi padta.

Reason 3 — Specific Behavior dena

Parent class ka method generic hota hai. Child class apna specific behavior de sakti hai bina baaki code ko chhede.

Interview Answer: Method overriding isliye use karte hain kyunki: (1) Runtime polymorphism achieve hota hai, (2) Code reuse hota hai, (3) Child class apna specific behavior de sakti hai.

3. Dynamic Method Dispatch

Definition: Jab parent class ka reference child class ke object ko point kare, aur method call runtime pe decide ho — ise Dynamic Method Dispatch kehte hain.

Dynamic = runtime pe decide hota hai (compile time pe nahi)

Dispatch = method call bhejna

Example — Kaise kaam karta hai?

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
```

```
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {

        Shape s;          // Parent reference

        s = new Circle(); // Runtime pe decide: Circle ka draw()
        s.draw();         // Output: Drawing a Circle

        s = new Rectangle(); // Runtime pe decide: Rectangle ka draw()
        s.draw();            // Output: Drawing a Rectangle
    }
}
```

Key Point — Compile Time vs Runtime

	Compile Time	Runtime
Reference type	Shape s	Shape s
Object type	Unknown	Circle / Rectangle
Which method called?	Shape ka draw()	Child ka draw()
Decided by	Compiler	JVM at runtime

Interview Tip: Dynamic Method Dispatch = Runtime Polymorphism. Dono same cheez hain — alag naam se poochha jaata hai.

4. Rules of Method Overriding

Ye rules yaad karo — koi bhi ek galat hua toh compile error aayega.

Rule 1 — Same Method Name

- Child class mein method ka naam bilkul same hona chahiye jaise parent mein hai.

Rule 2 — Same Parameters

- Parameters bhi bilkul same hone chahiye — same type, same count, same order.
- Agar alag parameters diye toh yeh Overloading ban jaata hai, Overriding nahi.

Rule 3 — Same Return Type (ya Covariant)

- Return type same hona chahiye.
- Covariant return type bhi allowed hai — matlab child class ka return type parent ke return type ka subclass ho sakta hai.

```
class Animal {
    Animal getInstance() { return new Animal(); }
}
class Dog extends Animal {
    @Override
    Dog getInstance() { return new Dog(); } // Covariant — allowed!
}
```

Rule 4 — Access Modifier: Kam Nahi Kar Sakte

- Child class mein method ka access kam nahi ho sakta, zyada ho sakta hai.

Parent Method	Child Method	Valid?
public	public	YES
public	protected	NO — access kam hua
protected	public	YES — access badha
protected	protected	YES
private	anything	NO — private override nahi hota

Rule 5 — static Method Override Nahi Hota

- static method ko override nahi kar sakte — usse Method Hiding kehte hain, Overriding nahi.

```
class Parent {
```

```
    static void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    static void show() { System.out.println("Child"); } // Hiding, NOT overriding
}
```

Rule 6 — final Method Override Nahi Hota

- Agar parent class ka method final hai toh child class use override nahi kar sakti — compile error aayega.

```
class Parent {
    final void display() { System.out.println("Can't override me!"); }
}
class Child extends Parent {
    void display() { } // ERROR: cannot override final method
}
```

Rule 7 — @Override Annotation use karo

- @Override likhna compulsory nahi hai, but best practice hai.
- Agar galti se method name typo ho gaya, @Override compiler ko error dikhayega — tabhi pata chalega.

Interview Tip: Rules yaad karne ka trick: Name, Parameters, Return type, Access, static nahi, final nahi, @Override lagao.

5. Overloading vs Overriding

Point	Overloading	Overriding
Class	Same class mein	Parent-Child class mein
Method Name	Same	Same
Parameters	DIFFERENT hone chahiye	SAME hone chahiye
Return Type	Alag ho sakta hai	Same hona chahiye
Inheritance	Zaroorat nahi	Zaroorat hai
Decide hota hai	Compile time pe	Runtime pe (JVM)
Polymorphism	Compile Time Polymorphism	Runtime Polymorphism
@Override	Nahi lagta	Lagana chahiye
private/static	Ho sakta hai	Nahi hota

Code mein Difference Dekho

```
// ────── OVERLOADING ──────
class Calculator {
    int add(int a, int b) { return a + b; }           // 2 params
    int add(int a, int b, int c) { return a+b+c; }    // 3 params — overloaded
    double add(double a, double b) { return a + b; }  // double type — overloaded
}

// ────── OVERRIDING ──────
class Animal {
    void sound() { System.out.println("Generic sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Woof!"); } // Same params — overriding
}
```

One-line Difference: Overloading = Same class, alag parameters. Overriding = Child class, same parameters.

Quick Revision — Interview ke liye

Topic	Ek Line Mein
Method Overriding	Child class mein parent ka method same signature se redefine karna
Why Overriding	Runtime polymorphism + code reuse + specific behavior
Dynamic Dispatch	Runtime pe JVM decide karta hai kaunsa method chalega
@Override	Compiler ko batata hai — yeh override hai, spelling check karo
Overloading	Same class, same name, alag parameters — compile time
Overriding	Child class, same name, same parameters — runtime

All the best for your interview!